

DASC7606

Deep Learning

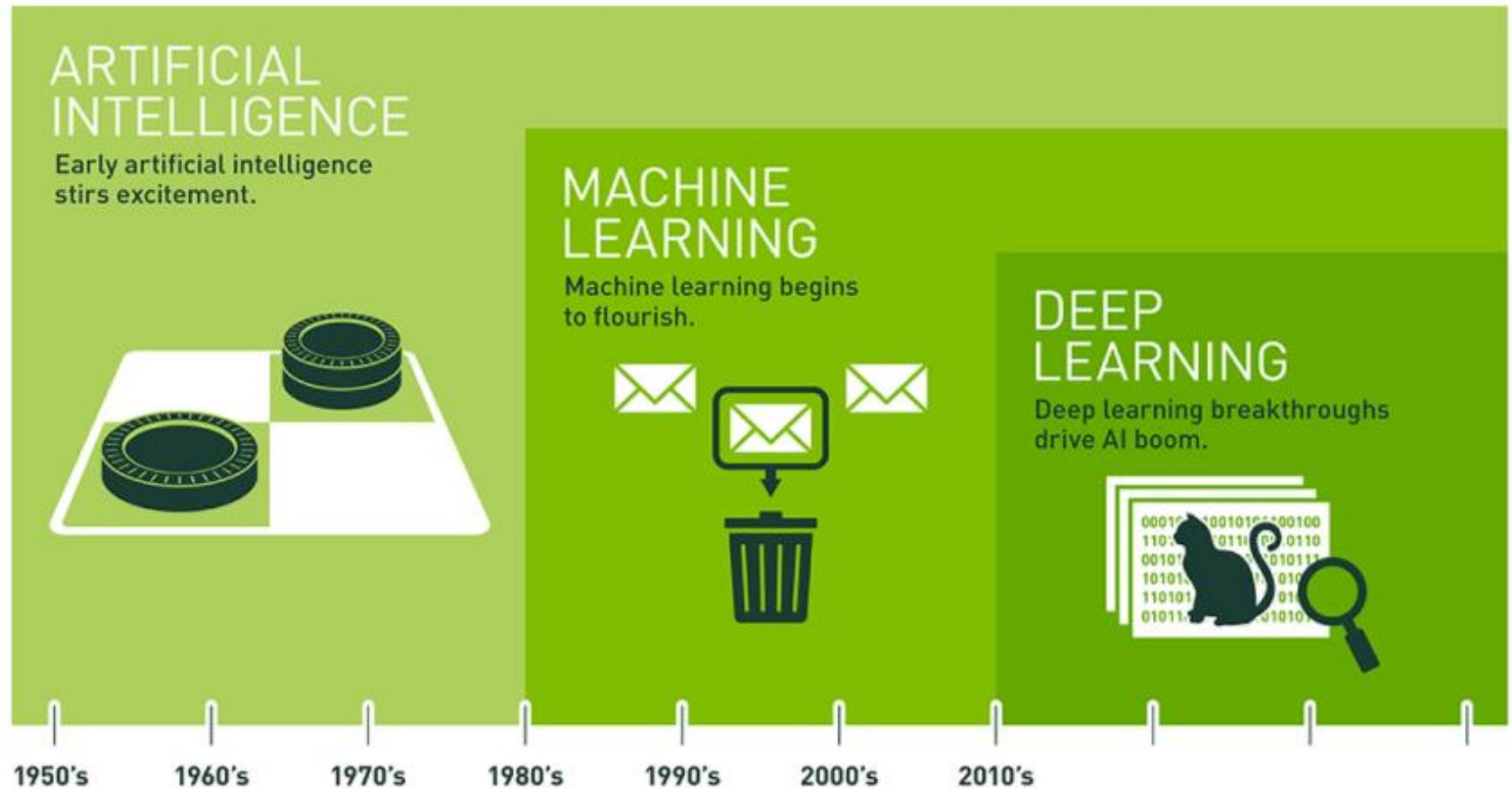
Perspectives

Dr. Bethany Chan
Professor Francis Chin
2026

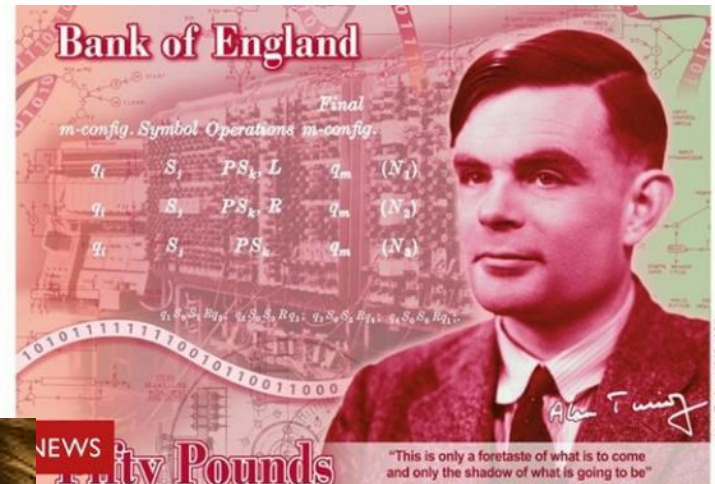
Perspectives – Outline (1)

1. History of AI
2. Turing Test: Imitation Game
3. Rule-Based Systems
4. Learning-Based Systems
5. Supervised Learning
6. Deep Learning vs Machine Learning
7. Representation Learning
8. The ABCD's of the Journey

1. History of AI

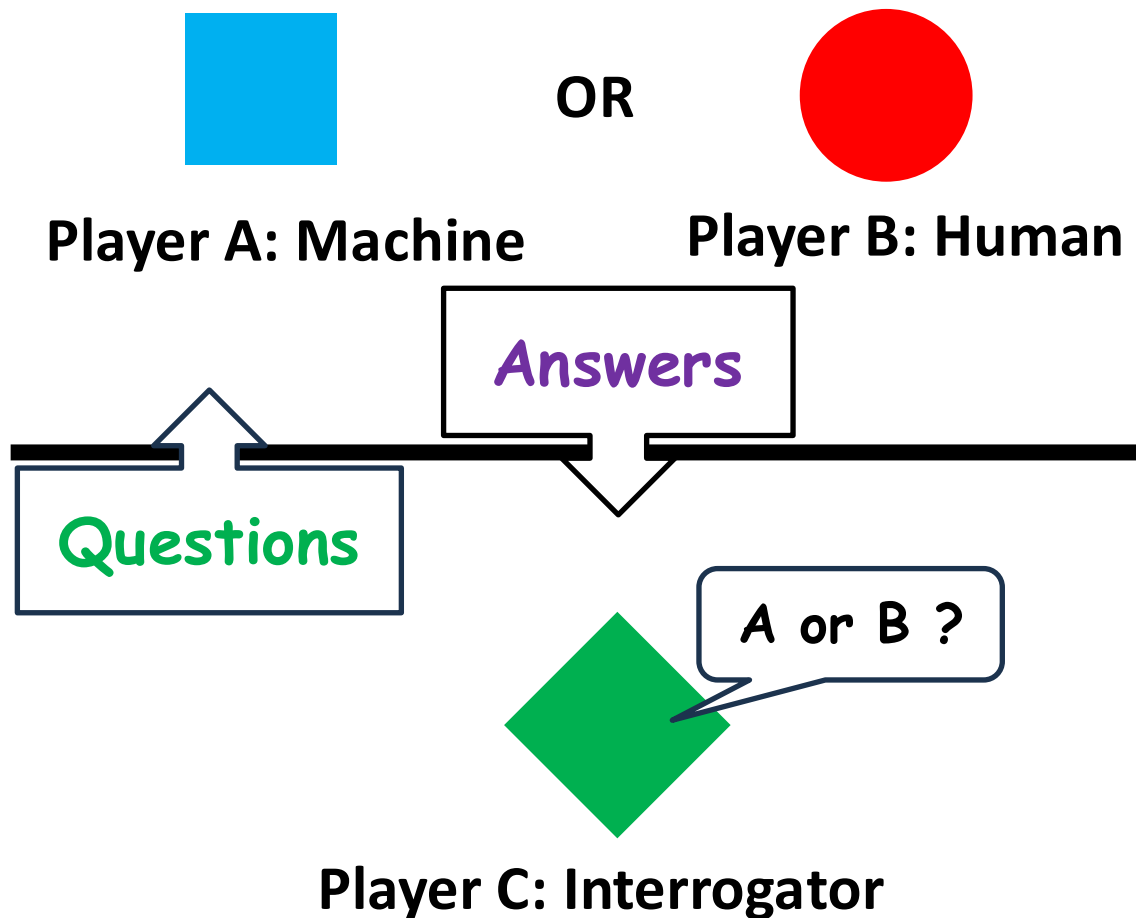


2. Turing Test: an imitation game



- Biographical film *Imitation Game* (2014)
WWII codebreaking using ENIGMA
- *Imitation Game* = Turing Test

Turing Test: an imitation game



To pass Turing Test:
Fool C into believing
he is interacting with
a human

(machine would have
to imitate a human).

If a machine passed
this test, it would be
considered intelligent.

Artificial intelligence
= Human intelligence
exhibited by machines

Turing Test: an imitation game

Welcome to

EEEEEE	LL	IIII	ZZZZZZZ	AAAAA
EE	LL	II	ZZ	AA AA
EEEEEE	LL	II	ZZZ	AAAAAAA
EE	LL	II	ZZ	AA AA
EEEEEE	LLLLLL	IIII	ZZZZZZZ	AA AA

Eliza is a mock **the first psychotherapy chatbot** in 1966.
The original program was written by Joseph Weizenbaum in 1966.
This implementation by Norbert Landsteiner 2005.

ELIZA: Is something troubling you ?

Same response to all answers

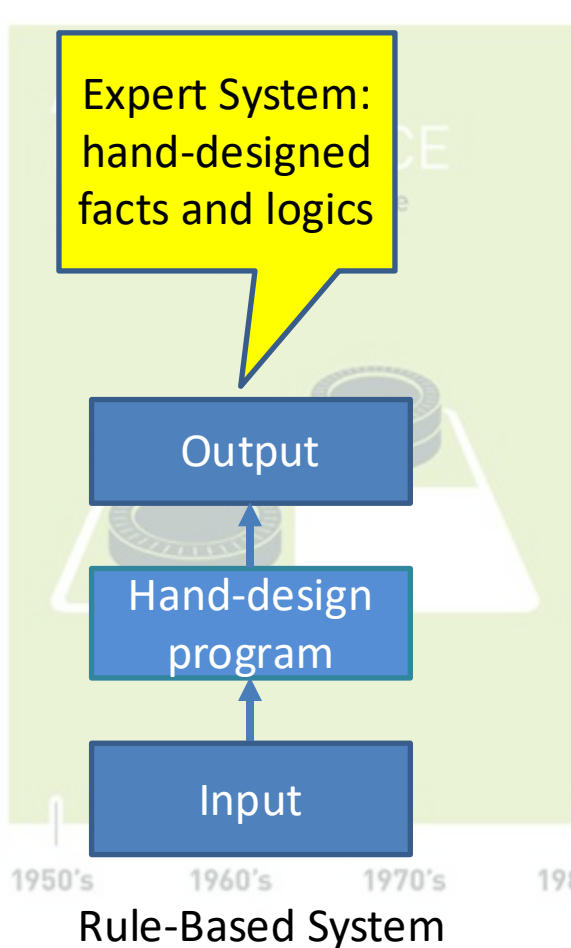
Spot the keyword
"always"

Type back

Spot "depressed"

Spot "unhappy"

3. Rule-Based Systems



Medical doctor

Simple Medical Diagnostics
(running nose, fever, headache,
coughing, sore throat, diarrhea,)

Bank Loan Approval

(income, gender, married, age,
own/rent property, previous loan,...)

Lawyer (legal advice)

Checkers ...

Require an expert (professional)
to define the input (features) and rules

4. Learning-Based Systems

“Instead of trying to produce a programme to simulate the adult mind, why not rather try to produce one which simulates the child's?

If this were then subjected to an **appropriate course of education** one would obtain the adult brain.”

- Alan Turing

→ **Start from a child's brain (simple program)**

Tabula rasa - idea that people are born without built-in mental content and therefore all knowledge is learnt

→ **Get more intelligent through time**

→ **Iterative improvement**

5. Supervised Learning



Asked **Questions**
given **Answers**

Questions & Answers

→ Practice exercises
(**training data**)

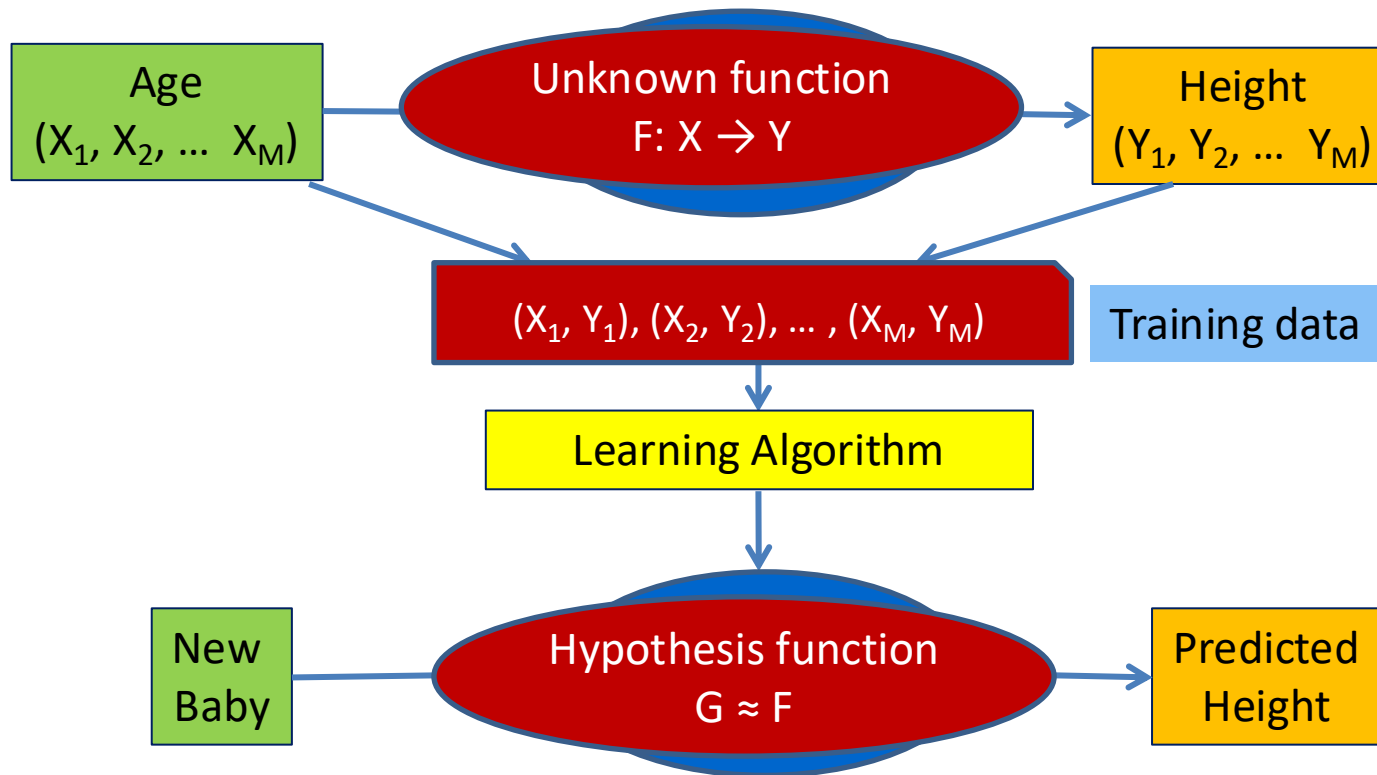
→ Mock exams (**validation**)

→ Final exams (**testing**)

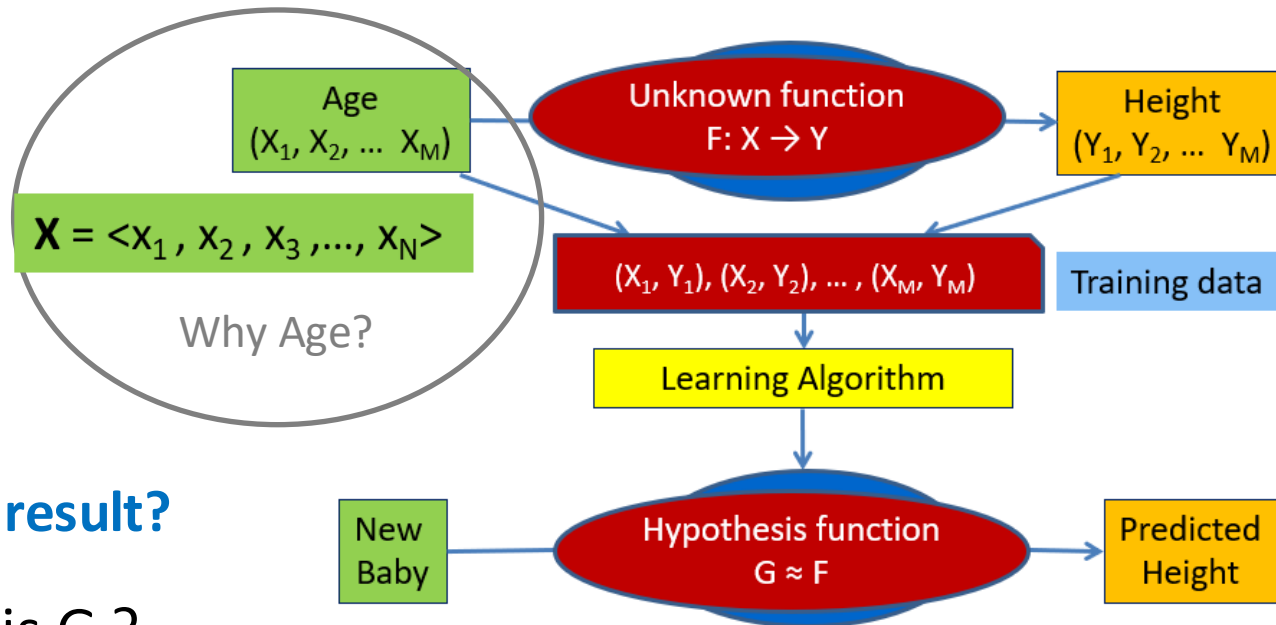
Supervised Learning: Model

Example: Given baby's age, what is the length of baby?

Age (months)	23	17	29	28	19	20	...
Height (cm)	80	75	83	82	78	77	...



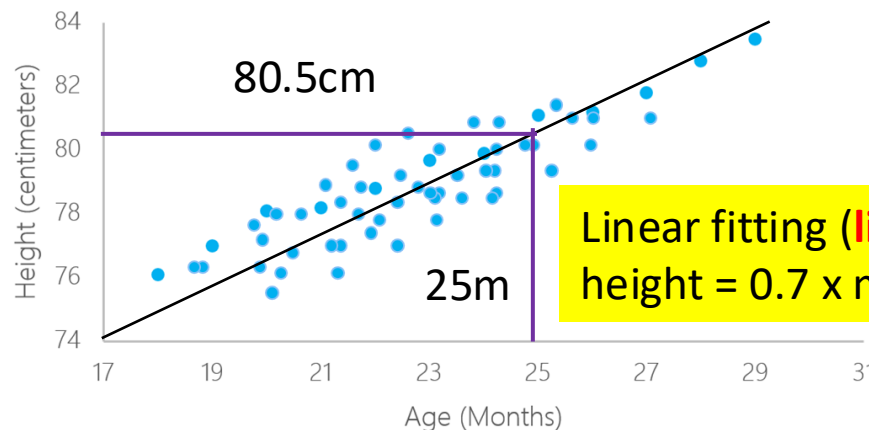
Supervised Learning: Model



How can we improve the result?

What is G ?

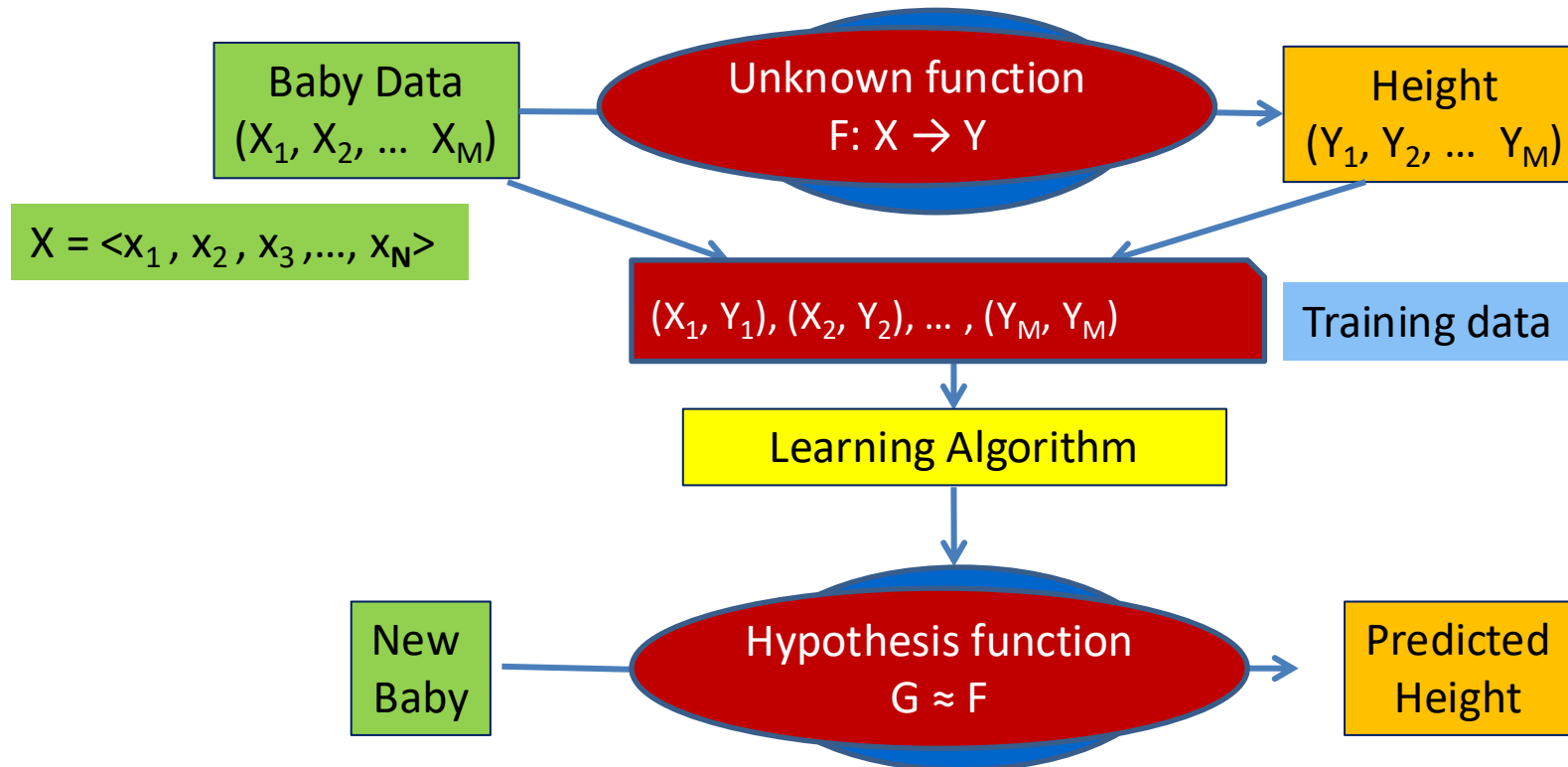
Baby Height vs Age



Linear fitting (**linear regression**)
height = 0.7 x months + 63 (cm)

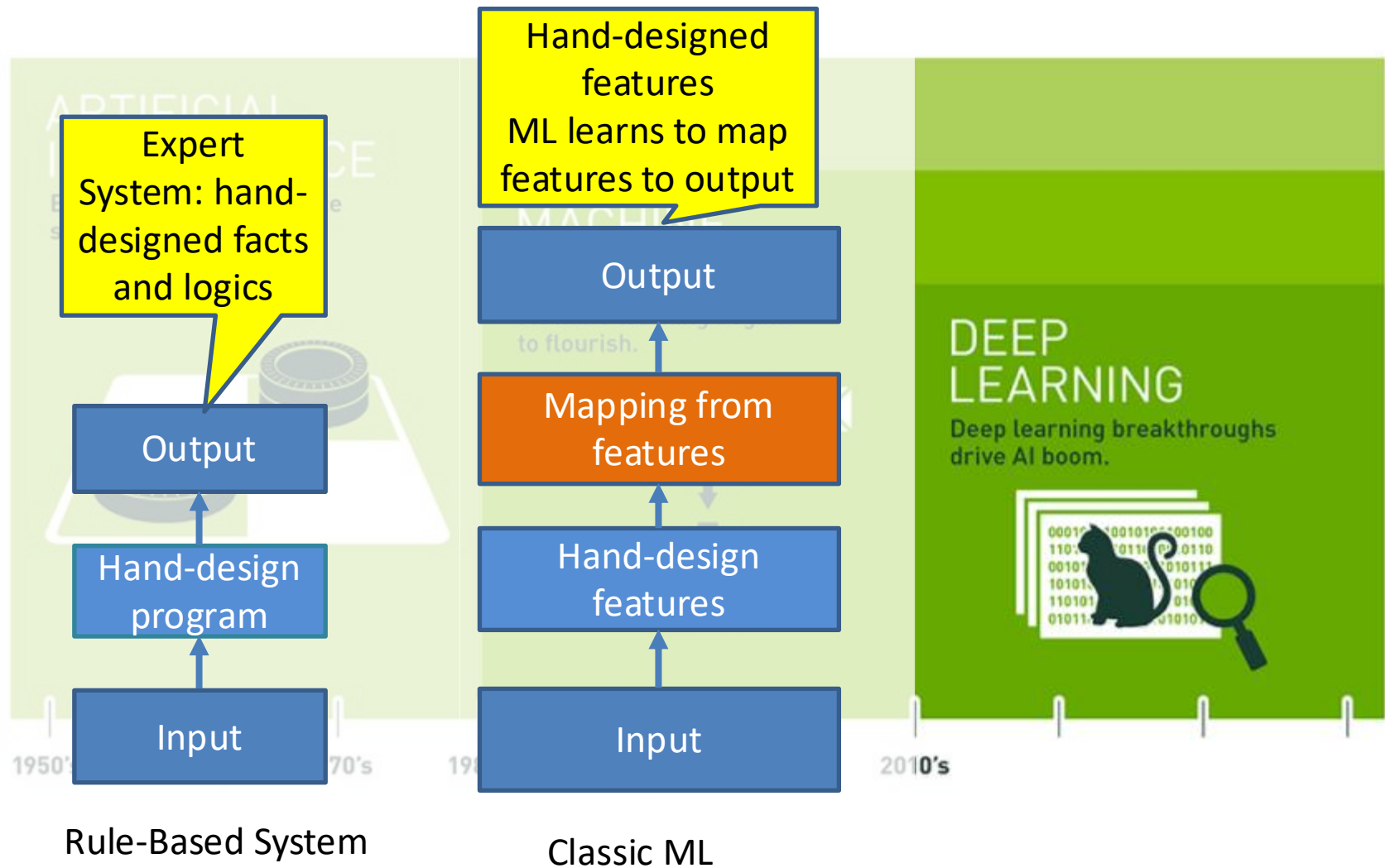
P.S. How to find the best line??? To be answered later

Supervised Learning: Model

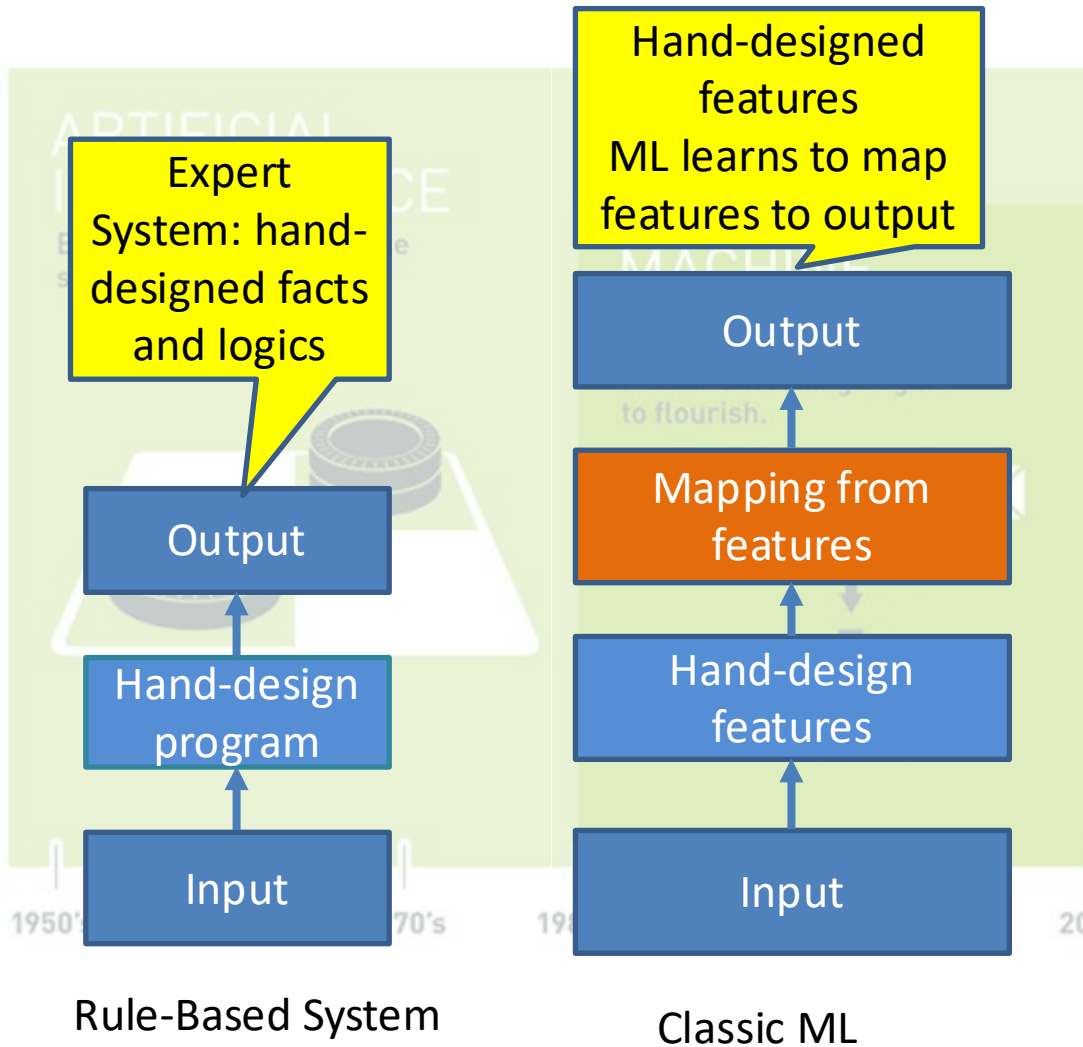


- What is G ?
- Example in previous slide: $\text{Height} = 63 + 0.7 * \text{age in months}$
- In general: $Y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_N x_N = \boldsymbol{\theta} \mathbf{X}$
- **Learn** values of θ_i 's (e.g. coefficients or weights for **age**, **gender**, **height of father**, **height of mother**, ...)

Learning-Based Systems: Machine Learning



Learning-Based Systems: Deep Learning



How do children learn?

6. Deep Learning vs Machine Learning

“Human beings **learn concepts starting from raw data**.

For instance, a child shown a few examples/ instances of a particular animal (like, say, cats) will soon learn to identify cats.

The learning process **does not involve a parent identifying features** (e.g., look at the whiskers or see the fur or the tail).

Human learning goes **from raw data to a conclusion** without the explicit step where features are identified and provided to the learner.”

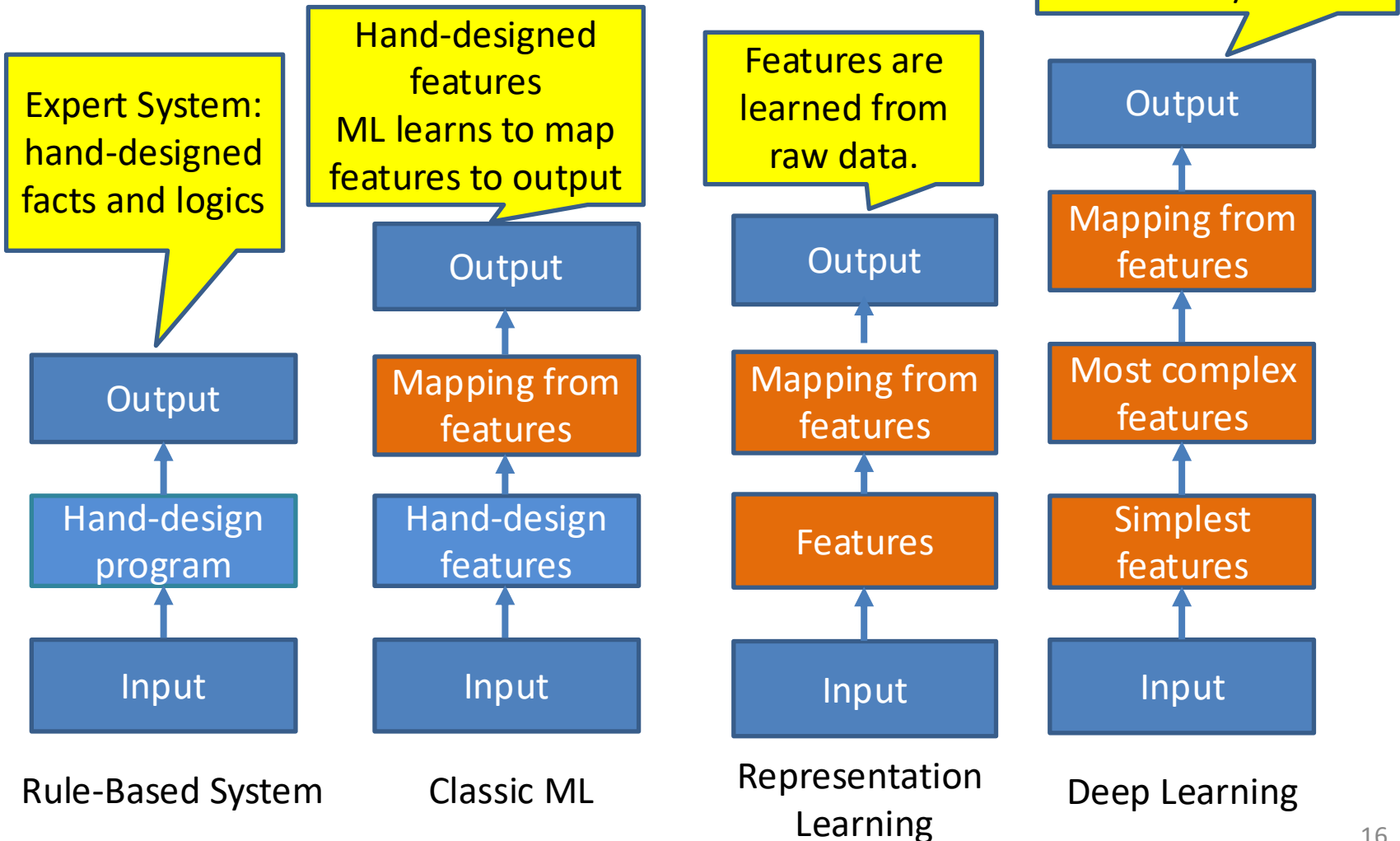
- *Deep Learning with Python: A Hands-on Introduction* by Nikhil Ketkar

→ **Learn from raw data**

7. Representation Learning

DL learns **multiple levels of representation** which corresponds to **multiple levels of abstraction**.

"Deep Learning: Theoretical Motivations" Yoshua Bengio 2015



8. The ABCD's of the Journey

A

lgorithms

- Efficient learning



Nature

<https://www.nature.com> > letters



Learning representations by back-propagating errors

by DE Rumelhart · 1986 · Cited by 40947 — We describe a new learning procedure, **back-propagation**, for networks of neurone-like units. The procedure repeatedly adjusts the weights of ...

- Open-source AI software (e.g. Tensorflow, PyTorch)

B

ig data

- ImageNet: ~15M images collected from web, labelled into ~22K categories
- Internet

C

omputing resources

- GPUs and cloud resources

D

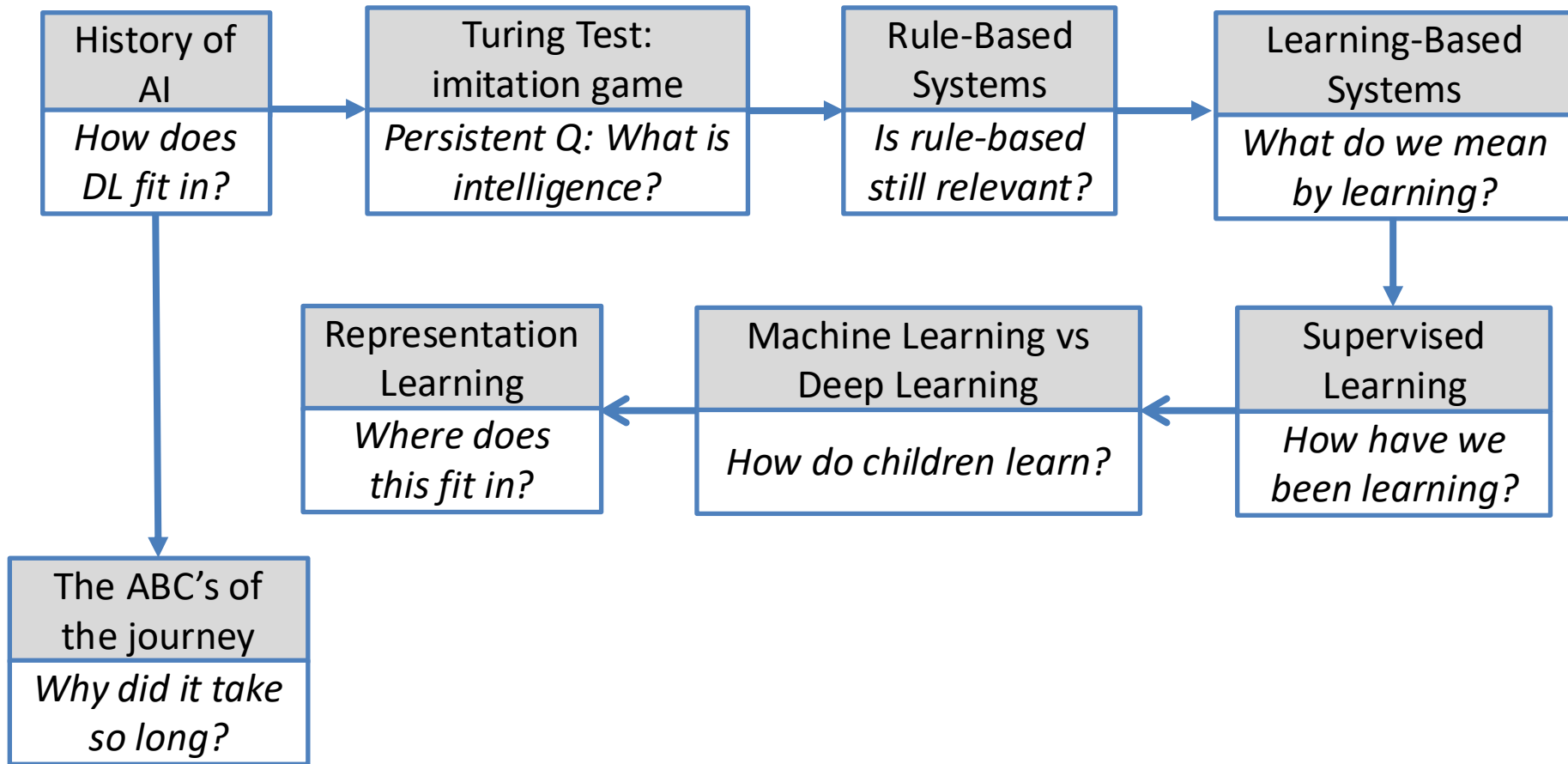
ollars

- Google, Meta, Microsoft, OpenAI...

What's D?



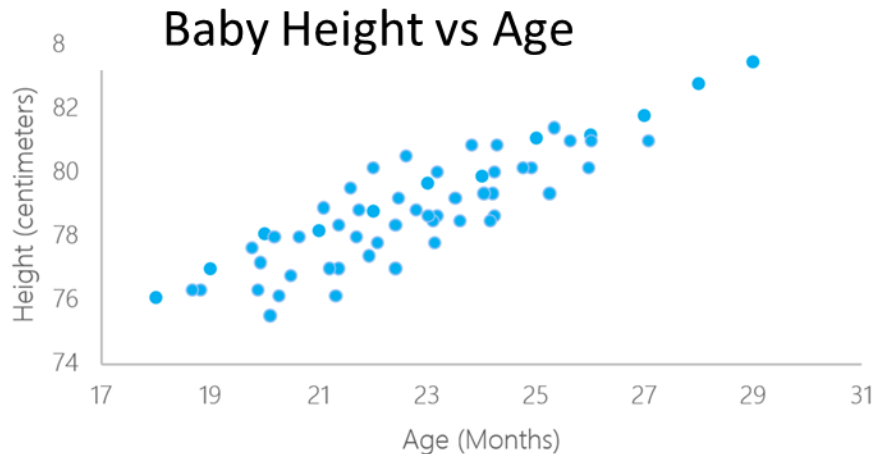
Perspectives – Mind Map (1)



Perspectives – Outline (2)

1. Supervised Learning
2. Cost Functions
3. Iterative Guessing
4. Gradient Descent
5. Regression vs Classification
6. Logistic Regression (for Binary Classification)
7. Multi-Classification

1. Supervised Learning: cost function



Predict $h(\mathbf{x})$ given $\mathbf{x} = (x_1, \dots, x_N)$

Find **hyperplane or line**:

$$h(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \dots + \theta_N x_N$$

[or $h_{\theta}(\mathbf{x}) = \boldsymbol{\theta} \mathbf{x}$ with $x_0 = 1$]

to minimize **error**

Loss function vs Cost function

A **loss function** is for a single training example (sometimes called **error function**).

A **cost function** $J(\boldsymbol{\theta})$ is the average loss over the entire training dataset, which is to be minimized.

2. Cost Function: mean square error

x	y
1.00	1.00
2.00	2.00
3.00	1.30
4.00	3.75
5.00	2.25

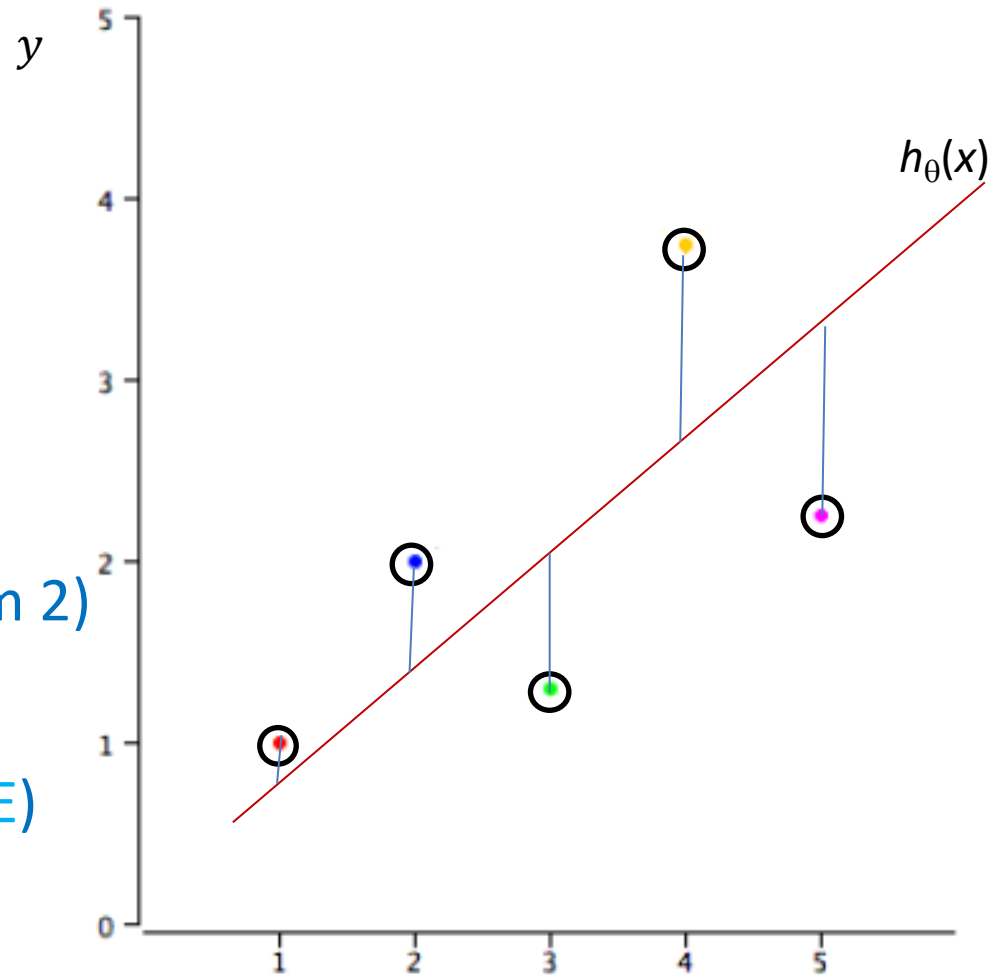
Mean square error (MSE) (norm 2)

$$= \frac{1}{M} \sum_{i=1}^M (h(x_i) - y_i)^2$$

$$\frac{1}{2M} \sum_{i=1}^M (h_{\theta}(x_i) - y_i)^2 \quad (\frac{1}{2} \text{ MSE})$$

Absolute Error Loss (norm 1)

$$= \frac{1}{M} \sum_{i=1}^M |h(x_i) - y_i|$$

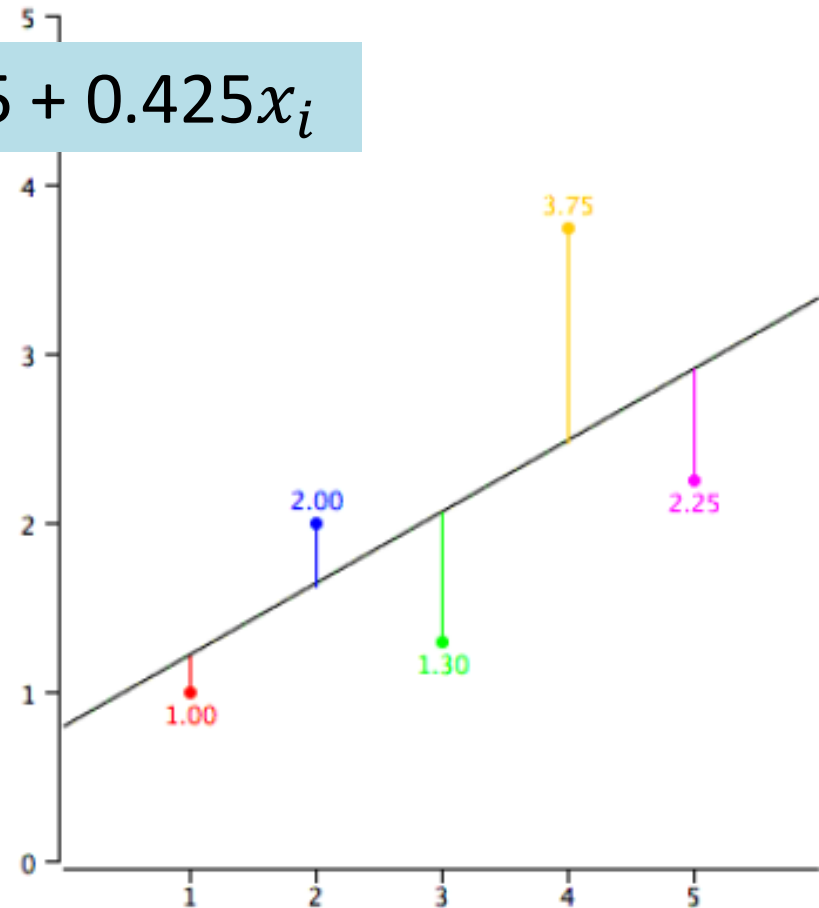


Cost Function: mean square error

Look at cost $J(\theta) = \frac{1}{M} \sum_{i=1}^M (h(x_i) - y_i)^2$ (norm 2)

where $h(x_i) = \theta_0 + \theta_1 x_i = 0.785 + 0.425x_i$

x	y	$h(x)$	$h(x) - y$	$(h(x) - y)^2$
1.00	1.00	1.210	0.210	0.044
2.00	2.00	1.635	-0.365	0.133
3.00	1.30	2.060	0.760	0.578
4.00	3.75	2.485	-1.265	1.600
5.00	2.25	2.910	0.660	0.436



3. Iterative Guessing

Make an initial guess for θ

Calculate the error $J(\theta)$

Repeat until error is small enough:

make a better guess for θ

calculate the error $J(\theta)$

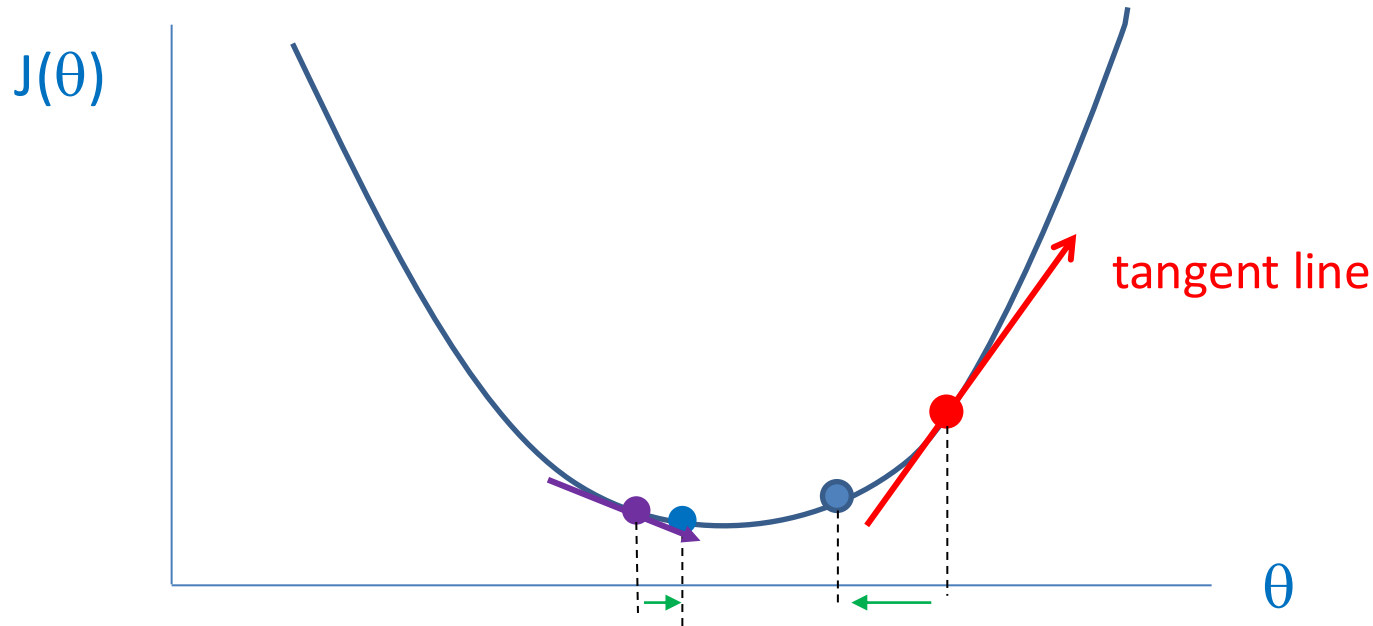
Return θ

Recall:

- Start from a child's brain (simple program)
- Get more intelligent through time
- iterative improvement

How to guess more intelligently?

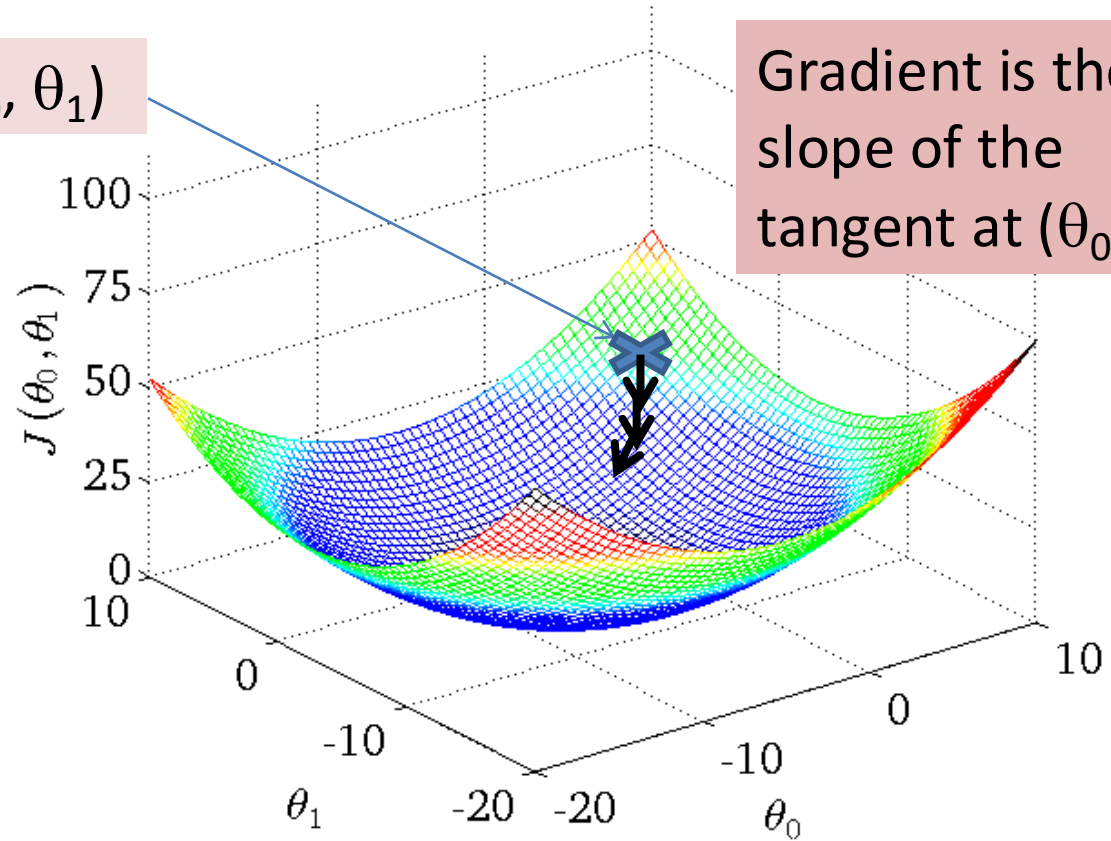
Iterative Guessing: gradient descent



- Gradient = slope of tangent at $\theta = \nabla J(\theta)$
- Next guess for θ : $\theta \leftarrow \theta - \alpha \nabla J(\theta)$ where $\alpha > 0$ is the learning rate
- If gradient is **up-sloping** ($\nabla J(\theta) > 0$), θ will **decrease**
- If gradient is **down-sloping** ($\nabla J(\theta) < 0$), θ will **increase**
- If tangent is **steep** ($|\nabla J(\theta)|$ is big), change will be **big**
- If tangent is **near horizontal** ($|\nabla J(\theta)|$ is near 0), change will be **small**

4. Gradient Descent (one feature)

Random guess of (θ_0, θ_1)



Repeat until error is small enough:

$$\theta_0 \leftarrow \theta_0 - \alpha \frac{\partial J(\theta_0, \theta_1)}{\partial \theta_0}$$

$$\theta_1 \leftarrow \theta_1 - \alpha \frac{\partial J(\theta_0, \theta_1)}{\partial \theta_1}$$

Gradient Descent (one feature)

$$J(\theta_0, \theta_1) = \frac{1}{2M} \sum_{i=1}^M (h_{\theta}(x_i) - y_i)^2 \quad \text{and} \quad h_{\theta}(x_i) = \theta_0 + \theta_1 x_i$$

Repeat until error is small enough:

$$\theta_0 \leftarrow \theta_0 - \alpha \frac{\partial J(\theta_0, \theta_1)}{\partial \theta_0} = \theta_0 - \alpha \frac{1}{M} \sum_{i=1}^M (h_{\theta}(x_i) - y_i)$$

$$\theta_1 \leftarrow \theta_1 - \alpha \frac{\partial J(\theta_0, \theta_1)}{\partial \theta_1} = \theta_1 - \alpha \frac{1}{M} \sum_{i=1}^M (h_{\theta}(x_i) - y_i) x_i$$

Gradient Descent (multiple features)

Repeat until error is small enough:

$$\theta_0 \leftarrow \theta_0 - \alpha \frac{1}{M} \sum_{i=1}^M (h_{\theta}(\mathbf{x}_i) - y_i) x_{i,0}$$

$$\theta_1 \leftarrow \theta_1 - \alpha \frac{1}{M} \sum_{i=1}^M (h_{\theta}(\mathbf{x}_i) - y_i) x_{i,1}$$

$$\theta_2 \leftarrow \theta_2 - \alpha \frac{1}{M} \sum_{i=1}^M (h_{\theta}(\mathbf{x}_i) - y_i) x_{i,2}$$

...

$$\theta_N \leftarrow \theta_N - \alpha \frac{1}{M} \sum_{i=1}^M (h_{\theta}(\mathbf{x}_i) - y_i) x_{i,N}$$

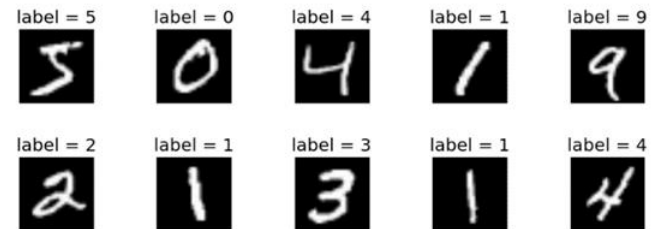
5. Regression vs Classification

Regression (continuous answer)

1. Predict the height of a baby
2. Predict the rent of a flat
3. Predict the probability of a person having a heart attack
4. Predict the probability of the handwritten digit is 0, is 1, is 2, ...

Classification (discrete answer)

3. Predict whether a person will have a heart attack or not (binary classification)
4. Given the image of a handwritten digit (0-9), determine which digit is it (multi-classification)



6. Logistic Regression (for Binary Classification)

Introduce the logistic function (sigmoid)

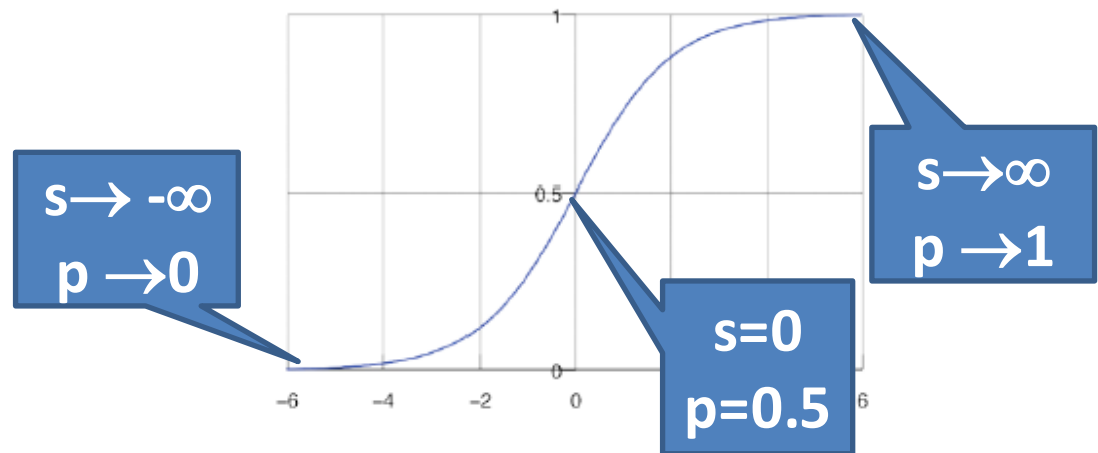
$\sigma: \mathbb{R}^N \rightarrow [0,1]$ interpreted as a probability

Helps to predict the **probability** of an event occurring (e.g. prediction of a heart attack)

Suppose $s = \theta x$ gives a “**risk score**” for the heart attack

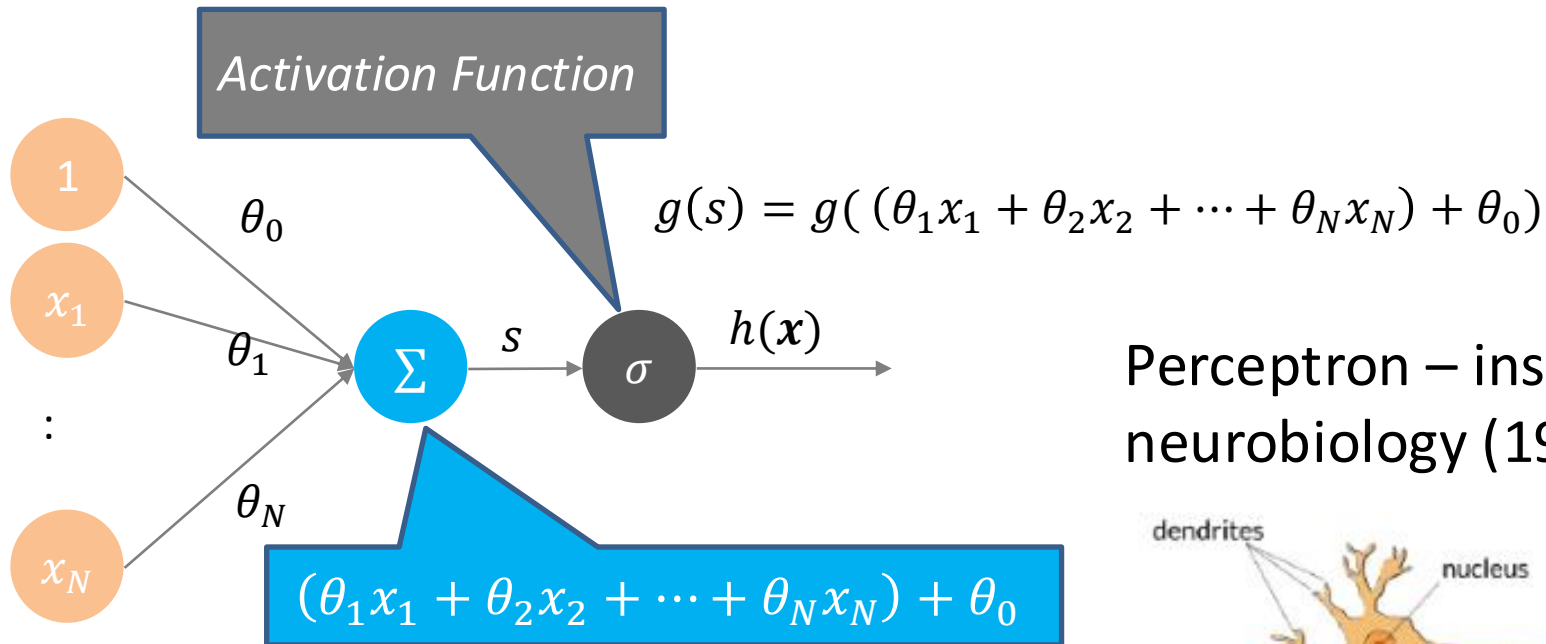
Prediction : $h(x) = \sigma(s) = \sigma(\theta x)$

$$\sigma(s) = \frac{e^s}{1 + e^s} = \frac{1}{1 + e^{-s}}$$

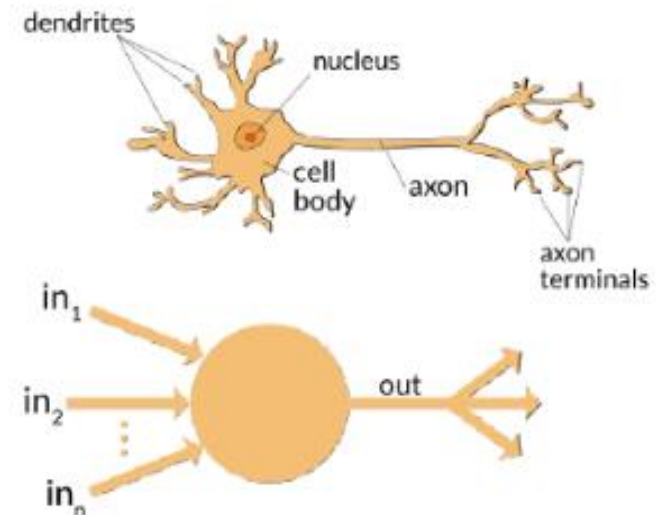


Logistic Regression, diagrammatically

Prediction : $h(\mathbf{x}) = \sigma(s) = \sigma(\boldsymbol{\theta}\mathbf{x})$



Perceptron – inspired by neurobiology (1954)

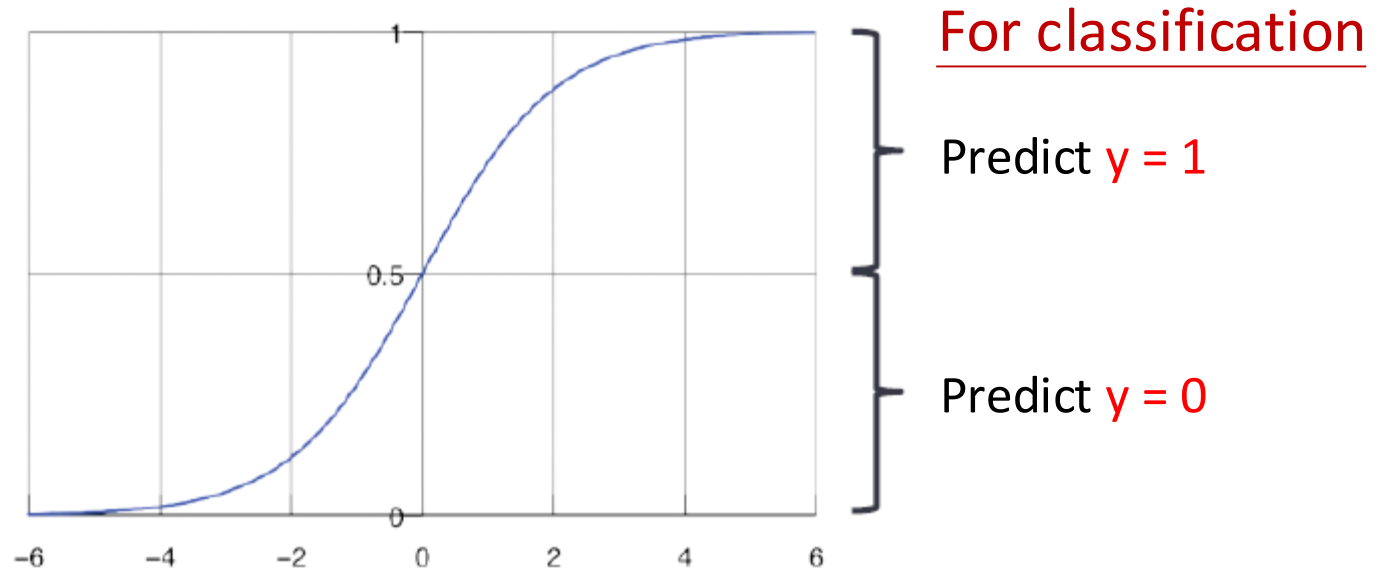


Biological neurons learn through processing information from environment, so as NN

- $\theta_0 = 0$ - no effect on the neuron,
- $\theta_0 > 0$ (+ve weight) - excitatory,
- $\theta_0 < 0$ (-ve weight) - inhibitory

Logistic Regression: binary classification

$$\sigma(s) = \frac{e^s}{1 + e^s} = \frac{1}{1 + e^{-s}}$$

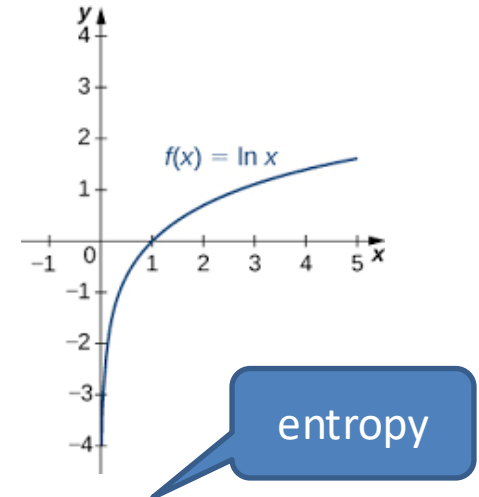


If the output > 0.5 (50% probability), i.e., $s > 0$,
it is classified as **positive class**.

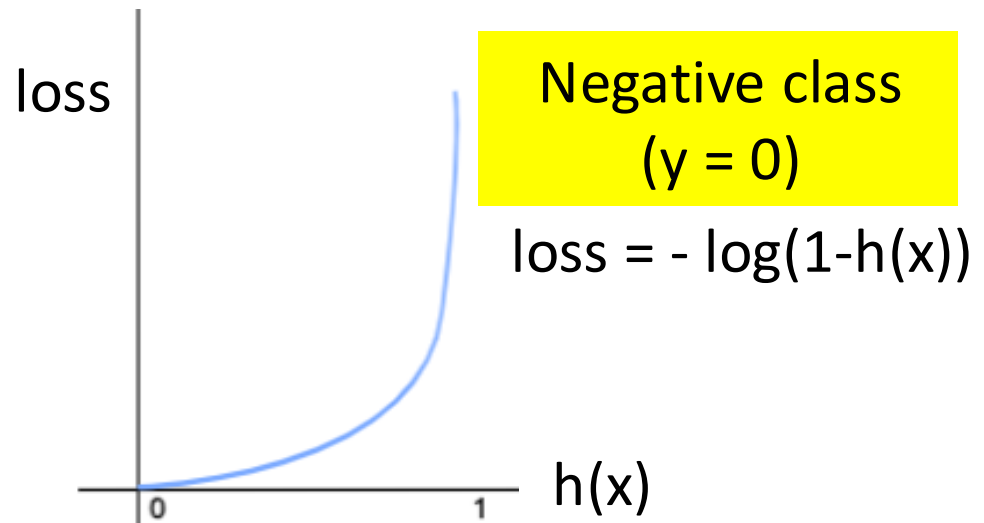
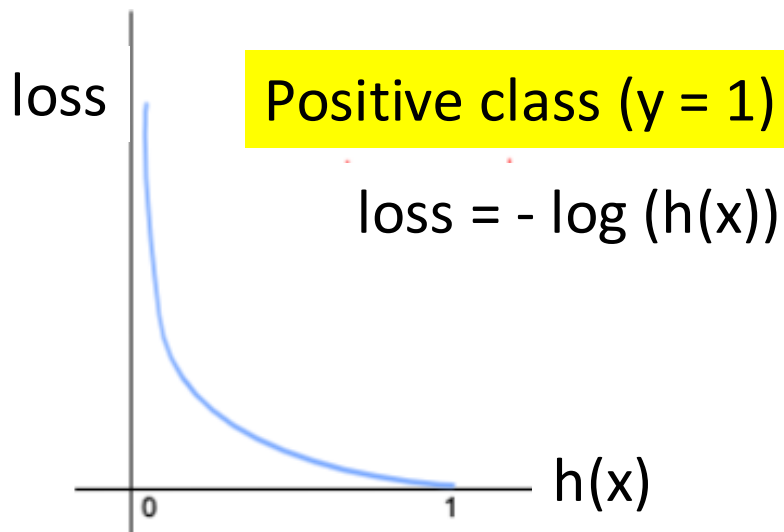
If the output < 0.5 , i.e., $s < 0$,
it is classified as **negative class**.

Logistic Regression: loss function

y	$h(x)$		
1	1	➔	Loss is 0
1	0	➔	Loss is very high
0	1	➔	Loss is very high
0	0	➔	Loss is 0



- To predict **positive class** ($y = 1$), $\text{loss} = -\log(h(x))$
- To predict **negative class** ($y = 0$), $\text{loss} = -\log(1 - h(x))$



Logistic Regression: loss function

$$J(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

$$J(h_{\theta}(x), y) = -y \log(h_{\theta}(x)) - (1 - y) \log(1 - h_{\theta}(x))$$

Remains when $Y = 1$

Removed when $Y = 0$

Remains when $Y = 0$

Removed when $Y = 1$

Gradient Descent (Logistic Regression)

$$J(\boldsymbol{\theta}) = -\frac{1}{M} \sum_{i=1}^M [y_i \log h(\mathbf{x}_i) + (1 - y_i) \log(1 - h(\mathbf{x}_i))]$$

$$\frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta}) = -\frac{1}{M} \sum_{i=1}^M \frac{\partial}{\partial \theta_j} \underbrace{[y_i \log h(\mathbf{x}_i) + (1 - y_i) \log(1 - h(\mathbf{x}_i))]}_{J_i(\boldsymbol{\theta})}$$

where $h(\mathbf{x}_i) = \sigma(s_i)$; $s_i = \mathbf{x}_i \boldsymbol{\theta}$, $\sigma(s_i) = \frac{1}{1 + e^{-s_i}}$

$s_i = \mathbf{x}_i \boldsymbol{\theta} = \theta_1 x_{i,1} + \dots \theta_j x_{i,j} + \dots \theta_N x_{i,N}$; $N = \text{dimension}$

$$\frac{\partial J_i(\boldsymbol{\theta})}{\partial \theta_j} = \frac{\partial J_i}{\partial h(\mathbf{x}_i)} \frac{\partial h(\mathbf{x}_i)}{\partial s_i} \frac{\partial s_i}{\partial \theta_j} \quad (\text{chain rule})$$

$$\begin{aligned} \frac{\partial J_i}{\partial h(\mathbf{x}_i)} &= - \left(\frac{y_i}{h(\mathbf{x}_i)} - \frac{1-y_i}{1-h(\mathbf{x}_i)} \right) \\ &= - \left(\frac{y_i - h(\mathbf{x}_i)}{h(\mathbf{x}_i)(1-h(\mathbf{x}_i))} \right) \end{aligned}$$

$$\frac{\partial}{\partial x} \log x = \frac{1}{x}$$

Gradient Descent (Logistic Regression)

$$J_i(\theta) = y_i \log h(x_i) + (1 - y_i) \log(1 - h(x_i))$$

where $h(x_i) = \sigma(s_i)$, $\sigma(s_i) = \frac{1}{1+e^{-s_i}}$

and $s_i = x_i \theta = \theta_1 x_{i,1} + \dots \theta_j x_{i,j} + \dots \theta_N x_{i,N}$

$$\frac{\partial J_i(\theta)}{\partial \theta_j} = \frac{\partial J_i}{\partial h(x_i)} \frac{\partial h(x_i)}{\partial s_i} \frac{\partial s_i}{\partial \theta_j} \quad (\text{chain rule})$$

$$1) \quad \frac{\partial J_i}{\partial h(x_i)} = \frac{y_i - h(x_i)}{h(x_i)(1 - h(x_i))}$$

$$2) \quad \frac{\partial h(x_i)}{\partial s_i} = \frac{e^{-s_i}}{(1 + e^{-s_i})^2}$$

$$= \frac{1}{1 + e^{-s_i}} \left(1 - \frac{1}{1 + e^{-s_i}} \right) = \sigma(s_i)(1 - \sigma(s_i))$$

$$3) \quad \frac{\partial s_i}{\partial \theta_j} = x_{i,j}$$

as $h(x_i) = \sigma(s_i)$

$$\frac{\partial J_i(\theta)}{\partial \theta_j} = (y_i - h(x_i)) x_{i,j}$$

$$\frac{\partial J(\theta)}{\partial \theta_j} = -\frac{1}{M} \sum_{i=1}^M (y_i - h(x_i)) x_{i,j}$$

Linear Regression

$$J(\theta) = \frac{1}{2M} \sum_{i=1}^M (h(x_i) - y_i)^2$$

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{M} \sum_{i=1}^M (h(x_i) - y_i) x_{i,j}$$

Repeat until error is small:

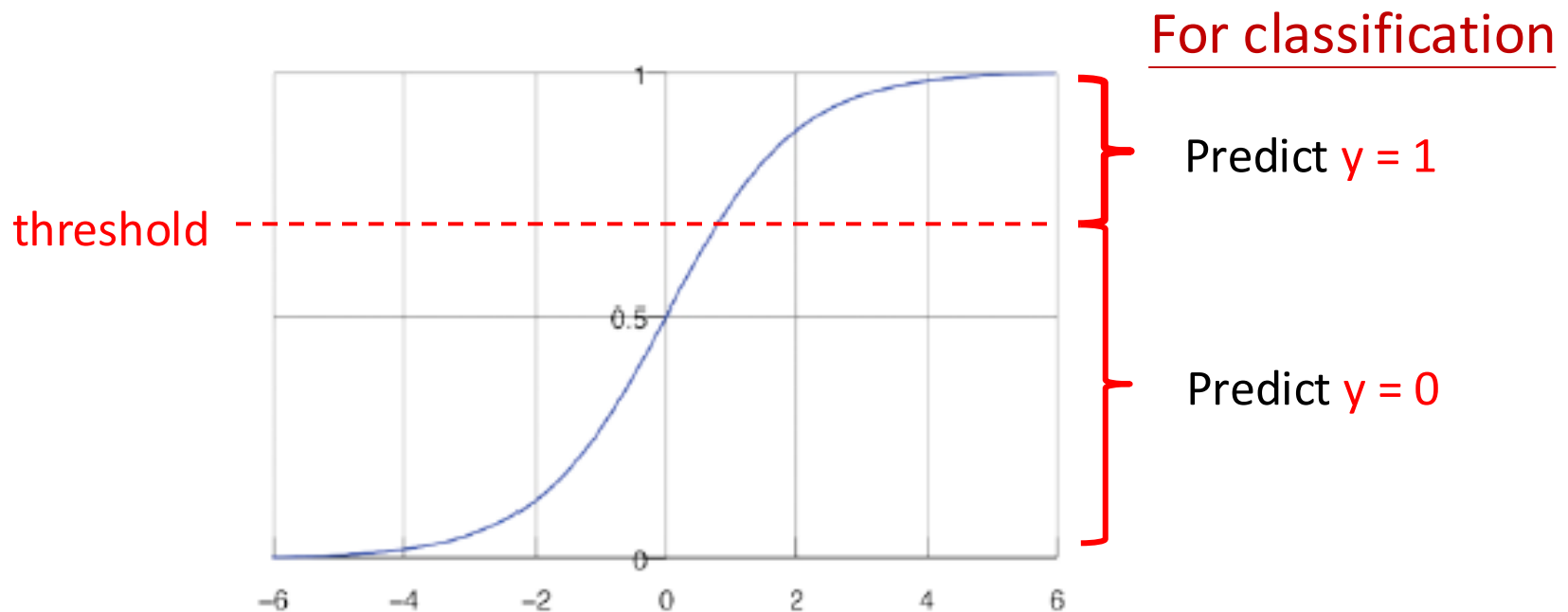
$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

for $j = 0, \dots, N$

Same equation as Linear Regression

Logistic Regression: binary classification

$$\sigma(s) = \frac{e^s}{1 + e^s} = \frac{1}{1 + e^{-s}}$$



Outcomes of Binary Classification

predict 1 if $f(x) > \text{threshold}$
predict 0 otherwise

	Actual 0	Actual 1
Predicted 0	true negative	false negative
Predicted 1	false positive	true positive

- **True positives:**
data points predicted as positive that are actually positive
- **False positives:**
data points predicted as positive that are actually negative
- **True negatives:**
data points predicted as negative that are actually negative
- **False negatives:**
data points predicted as negative that are actually positive

Logistic Regression: threshold for precision

predict 1 if $f(x) > \text{threshold}$
predict 0 otherwise

	Actual 0	Actual 1
Predicted 0	true negative	false negative
Predicted 1	false positive	true positive

$$\text{precision} = \frac{\text{true positives}}{\text{predicted positives}} = \frac{\text{true positives}}{\text{false positives} + \text{true positives}}$$

Higher threshold $\Rightarrow$ more difficult to be positive $\Rightarrow$ less false positive

↑ threshold	↑ precision
↓ threshold	↓ precision

Prosecution of criminals – want higher precision (want lower false positives)
(if you predict someone is a criminal, **you don't want to be wrong**)

Logistic Regression: threshold for recall

predict 1 if $f(x) > \text{threshold}$
predict 0 otherwise

	Actual 0	Actual 1
Predicted 0	true negative	false negative
Predicted 1	false positive	true positive

$$\text{recall} = \frac{\text{true positives}}{\text{actual positives}} = \frac{\text{true positives}}{\text{false negatives} + \text{true positives}}$$

Higher threshold $\Rightarrow$ more negatives $\Rightarrow$ more false negative

↑ threshold	↑ precision	↓ recall
↓ threshold	↓ precision	↑ recall

Disease screening – want higher recall (want lower false negatives)
(if you predict someone doesn't have the disease, **you don't want to miss**)

Logistic Regression: threshold for F1

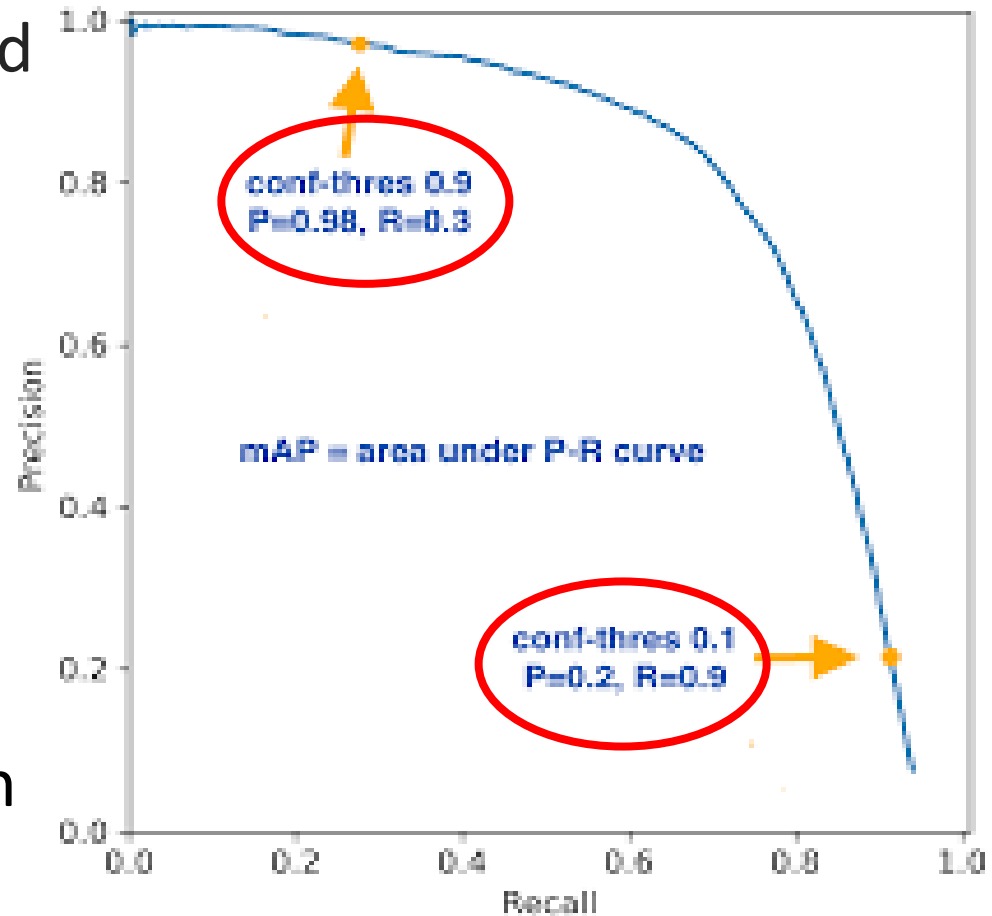
- Ideally both precision and recall are 1
- With different thresholds, we can have higher precision and recall values
- Depending on applications
 - Disease screening – higher recall (don't want to miss)
 - Prosecution of criminals – higher precision (don't want to be wrong)
 - Identify terrorists - both

$$F1 = 2 * \frac{\text{precision} * \text{recall}}{\text{precision} + \text{recall}}$$

- Harmonic mean instead of simple average to penalize extreme values

Precision-Recall Curve (PR curve)

- Precision-recall curve shows the tradeoff between precision and recall for different threshold.
- A model with high threshold (high P and low R) returns very few results, but most of them are correct.
- A model with low threshold (low P and high R) returns many results, but most of them will be incorrect.
- An ideal model will have high P and high R and will return many results with all correctly predicted
- A high area under curve (AUC) represents both high recall and high precision.

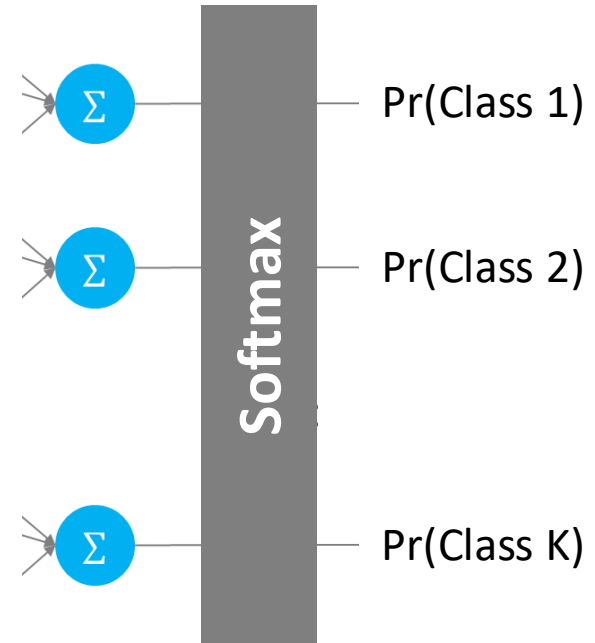
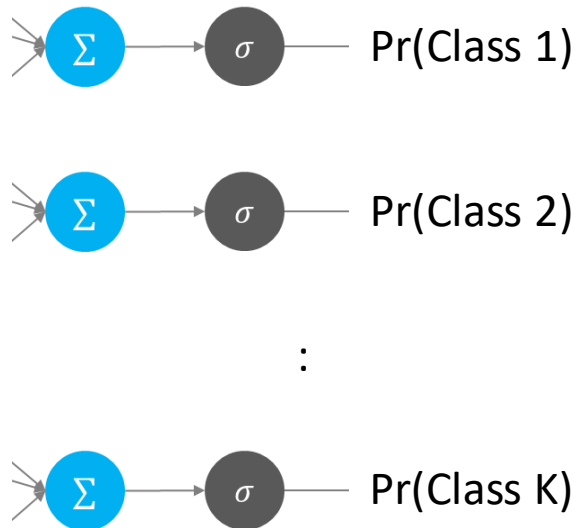


7. Multi-Classification: MNIST

MNIST (Mixed National Institute of Standards and Technology) database



Multi-Classification: softmax



$\text{Pr}(\text{Class } x)$ = Probability of Class x being the correct class
Class with **highest** probability is the predicted class

Softmax function: not only normalizes a set of scores to numbers in $[0,1]$ but also makes sure that the numbers all add up to 1

Multi-Classification: softmax vs sigmoid

For **sigmoid** (2-class):

Class 1 probability: $\sigma(s) = \frac{1}{1+e^{-s}}$

Class 2 probability: $1 - \sigma(s) = \frac{e^{-s}}{1+e^{-s}} = \frac{1}{1+e^s}$

For **softmax** (K -class):

Given scores $s_1, s_2, \dots, s_K$

Class k probability: $\frac{e^{s_k}}{\sum e^{s_i}}$

For **softmax** (2-class):

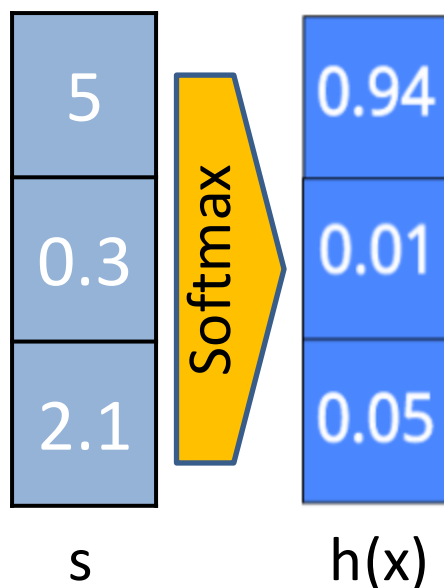
Class 1 probability: $\frac{e^{s_1}}{e^{s_1}+e^{s_2}} = \frac{1}{1+e^{s_2-s_1}}$

Class 2 probability: $\frac{e^{s_2}}{e^{s_1}+e^{s_2}} = \frac{1}{1+e^{s_1-s_2}}$

Multi-Classification: softmax



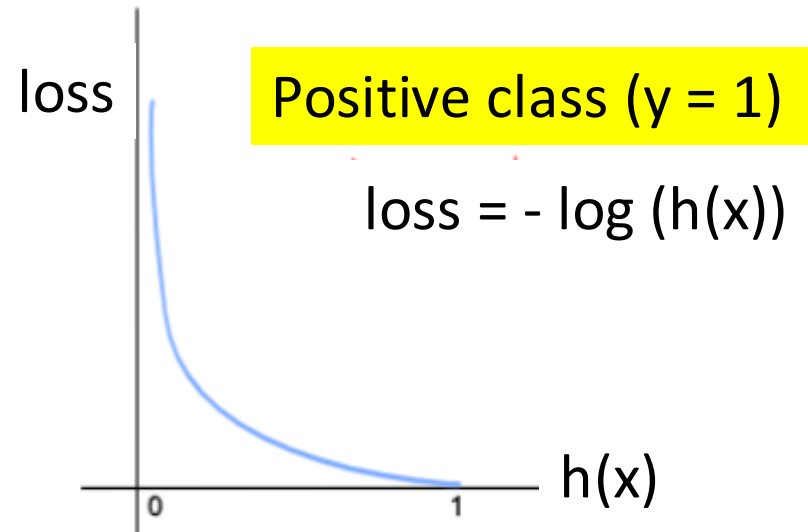
- **Question:** Why do we have to pass each value through an exponential before normalizing them? Why can't we just normalize the values themselves?
- **Answer:** This is because the goal of softmax is to make sure **one value is very high** (close to 1) and all **other values are very low** (close to 0).
- Softmax uses exponential to make sure this happens.



Multi-Classification: Loss Function

Categorical Cross Entropy

$$\begin{aligned} \text{Loss}(h(x), y) \\ = - \sum_c y \log(h(x)) \end{aligned}$$



Ex:

Predicted $h(x)$	Label y
[0.94, 0.01, 0.05]	[0, 1, 0]

$$\text{Loss} = -[0 * \log(0.94) + 1 * \log(0.01) + 0 * \log(0.05)] = -\log(0.01)$$

Perspectives – Mind Map (2)

